

Fourier Neural Operator for 1D Burgers' Equation

- Objective: Learn the solution operator instead of a single PDE solution
- Operator: $G: u_0(x) \rightarrow u(x,T)$
- PINN: retrain for each new initial condition
- FNO: train once, predict many unseen initial conditions

PINN:

- Retrained from scratch for every new initial condition
- One network $\approx$ one solution
- New IC $\rightarrow$ new optimization problem

FNO:

- Trained once across a distribution of ICs
- Learns the solution operator $G: u_0 \rightarrow u(x,T)$
- New IC $\rightarrow$ single forward pass, no retraining

Dataset Generation

- PDE: $u_t + uu_x = \nu u_{xx}$
- Gaussian Random Field initial conditions
- Numerical Burgers solver(using a random ν from the range[0.01,0.1])
- 1000 training trajectories, 200 test trajectories
- Input: $u_0(x)$, Output: $u(x,T)$

Gaussian Random
Field



Compute PDE
of Burger's
equation



Numerically
Solve Burger's
equation

FNO Architecture

- Input $u_0(x)$ + v channel
- Lifting layer
- 4 Fourier layers (16 modes, hidden channels = 64)
- Projection layer
- Loss: MSE, Optimizer: Adam



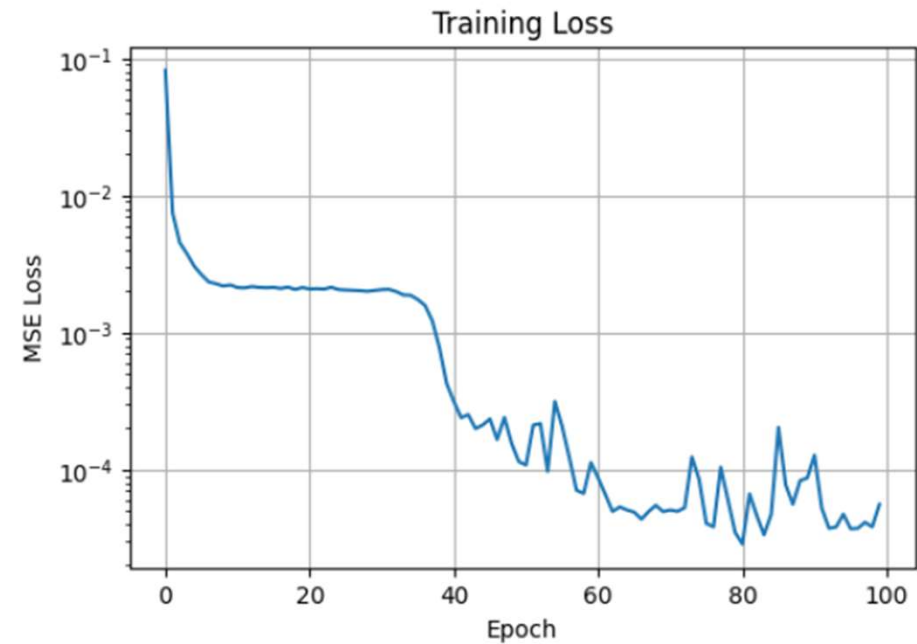
Training

- 1000 training samples
- Batch size: 32
- Epochs: 100
- Learning rate: 0.001

```
batch_size = 32

train_loader = DataLoader(
    TensorDataset(X_train, Y_train),
    batch_size=batch_size,
    shuffle=True,
)

test_loader = DataLoader(
    TensorDataset(X_test, Y_test),
    batch_size=batch_size,
    shuffle=False,
)
```



Results

- 200 held-out test samples
- Relative L^2 Error $\approx 2.8\%$
- Good agreement with ground truth

```
model.eval()

predictions = []

with torch.no_grad():
    for x_batch, _ in test_loader:
        x_batch = x_batch.to(device)
        pred = model(x_batch)

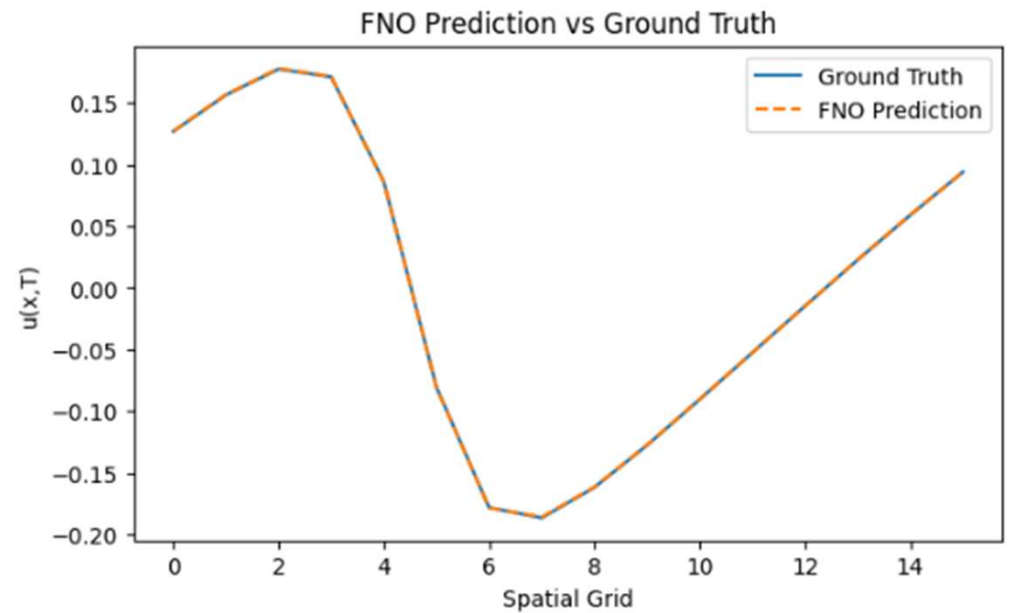
        predictions.append(pred.cpu())

predictions = torch.cat(predictions, dim=0)

error = torch.norm(predictions - Y_test) / torch.norm(Y_test)

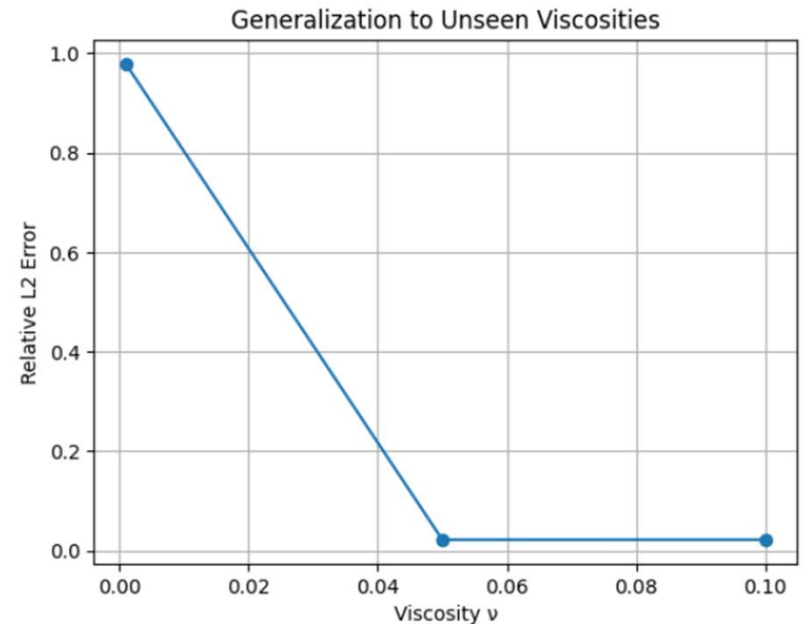
print("Relative L2 Error =", error.item())

Relative L2 Error = 0.02837222557663918
```



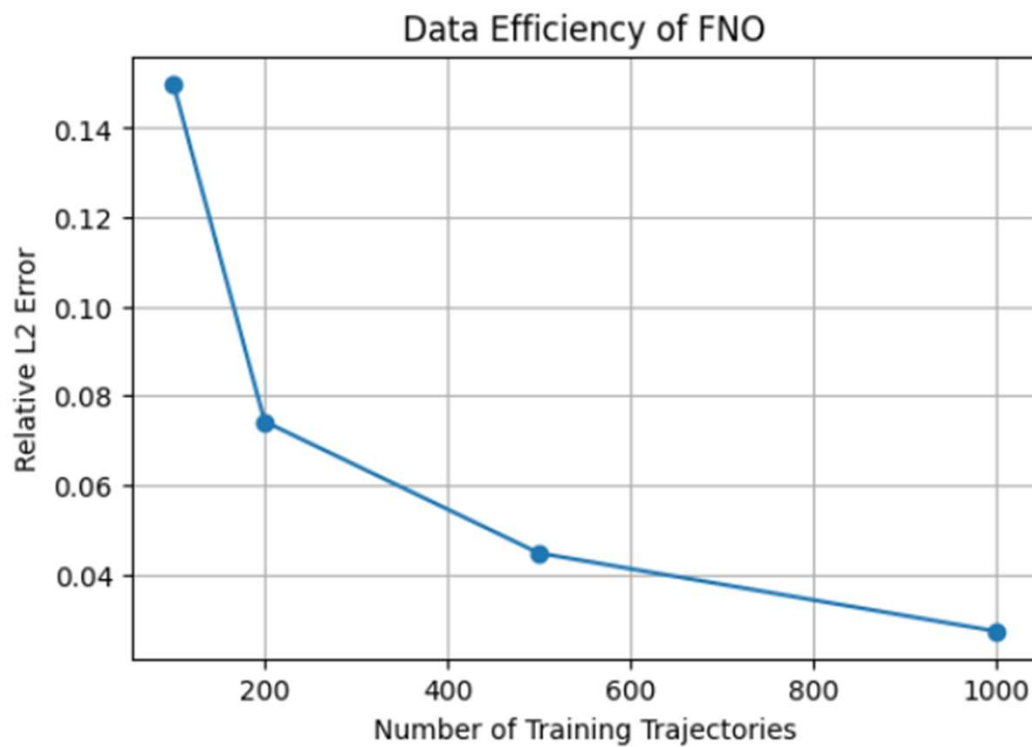
Generalization

- Train: $\nu = (0.01, 0.1)$
- Test: $\nu = 0.001, 0.05, 0.1$
- For $\nu=0.05, 0.1$ Error holds at 2.17% — matching the overall test error. The ν -conditioning channel lets the model interpolate within its trained range
- For $\nu=0.001$, Error spikes to ~98%. This is extrapolation, not a model bug — at this ν the PDE forms near-shocks the reference solver itself can't resolve cleanly (see Limitations).



Data Efficiency

Training Samples	Relative L ² Error
100	14.99%
200	7.43%
500	4.48%
1000	2.73%



Limitations



Extrapolation outside training ν -range

At $\nu = 0.001$ (below the trained $U(0.01, 0.1)$ range), relative error jumps to $\sim 98\%$. The FNO has never seen this regime and simply doesn't know how to condition on it.



Reference solver quality at low ν

At $\nu = 0.001$, Burgers' equation develops near-shock gradients. The central-difference solver ($N_x = 128$, no upwinding) likely can't resolve this stably so part of the "error" may be a bad ground truth, not just a bad prediction.

What I would try Next



Widen the training v -range

Include lower v (e.g. down to 0.001–0.005) during training so the model has actually seen near shock behavior, instead of extrapolating to it.



Upgrade the reference solver

Use an upwind or shock capturing scheme (or finer grid) for low- v trajectories, so the ground truth itself is trustworthy before blaming the FNO.



Run the actual PINN benchmark

Complete a real timed PINN baseline (train + solve, per instance) with FNO.



Compare with DeepONet

Training DeepONet and comparing it with FNO under same conditions like number of training trajectories, number of epochs

When Do Operators Win?

- Many instances of the same PDE family (different ICs, parameters) → train the operator once, evaluate instantly, repeatedly
- Real time or interactive use (design loops, control) where per-query PINN training is far too slow
- A single one-off PDE solve with no reuse → a PINN's zero pre-training cost wins
- Regimes far outside the training distribution (as seen at $v = 0.001$) → the operator needs to have actually seen that regime to be reliable